CSCI 2210 Checkpoint 1

Eric Johnson, Abimael Lozano, Manasi Movva

March 30, 2026

1 UML Class Diagram

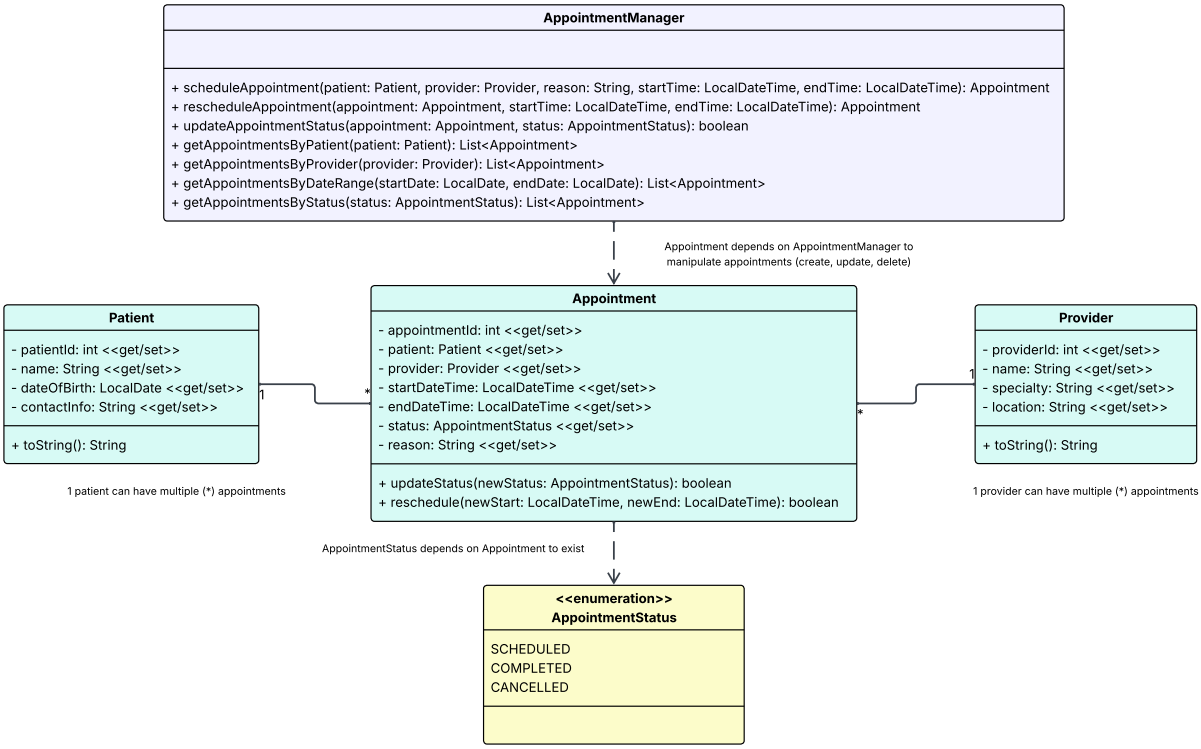

Figure 1: UML Class Diagram for core classes, program logic class, and enum class

Relationships Between Classes

The primary class is the Appointment class, which handles the appointment information that relates a patient to a provider. In order for an appointment to exist, it must first be evaluated logically through the AppointmentManager class, hence the dashed line with an open head. This similar relationship is represented between the AppointmentStatus enum and Appointment classes. In order for the AppointmentStatus object to exist, an appointment must be scheduled. The Patient and Appointment classes represent an association where (1) Patient can have multiple (*) appointments scheduled. The same relationship applies to the Appointment and Provider classes, where (1) Provider can have multiple (*) appointments also scheduled.

2 Class Descriptions

The core classes consist of the Patient, Provider, and Appointment classes. These can be represented (Figure 2) in UML format as follows:

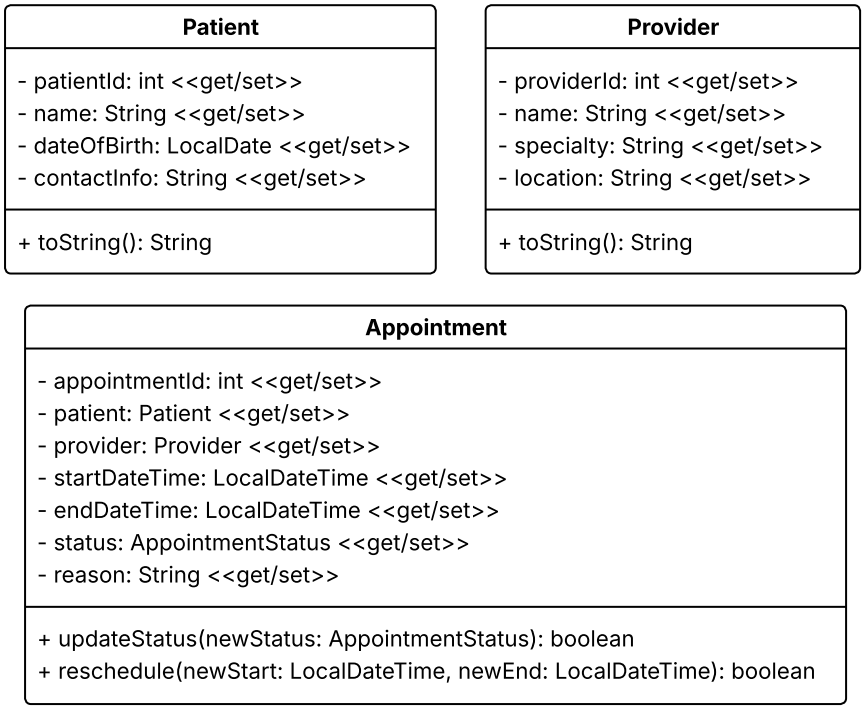

Figure 2: Core classes: Patient, Provider, Appointment UML representation

Patient Class Description

The Patient class represents an individual who schedules appointments with a provider for healthcare services. Every patient has a unique `patientId` used to identify them. The class contains details about the patient including name, date of birth, and contact information. The Patient class includes getter and setter methods for safe access and modification of patient data. Included is a `toString()` method that returns a readable string of the patient's information.

Provider Class Description

The Provider class represents a healthcare professional who receives scheduled appointments from patients. The system ensures that appointments for the same provider do not overlap. Each provider is identified with a unique `providerId.` The class contains details about the provider. Including name, medical specialty, and location where they practice. The Provider class includes getter and setter methods for attribute management. Included is a `toString()` method that returns a readable string of the provider's information.

Appointment Class Description

The Appointment class represents an appointment between a patient and a provider scheduled for a given time. Each appointment has a unique `appointmentId`. It also references both a Patient object and a Provider object. The class stores information related to scheduling, including the start date and time, end date and time, the reason for the visit, and the appointment status. The Appointment class includes getter and setter methods for managing its instance fields/information. Also included are behavior methods such as `updateStatus()`, which changes the appointment's status, and `reschedule()`, which updates the appointment time.

The AppointmentStatus enum class (Figure 3) UML representation is as follows:

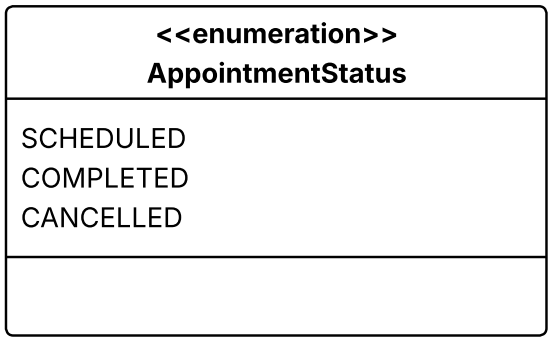

Figure 3: Enum class UML representation

Appointment Status enum Class Description

The AppointmentStatus enum defines the possible states of an appointment. Using an enum ensures that the appointment's state can be set to a valid predefined state, such as:
`SCHEDULED` – the appointment has been made and scheduled for a future date/time
`COMPLETED` – the appointment has already taken place
`CANCELLED` – the appointment has been cancelled and will not take place
AppointmentStatus depends on Appointment to exist.

AppointmentManager Program Logic Class Description

The AppointmentStatus class, responsible for core program logic can, be represented (Figure 4) as follows:

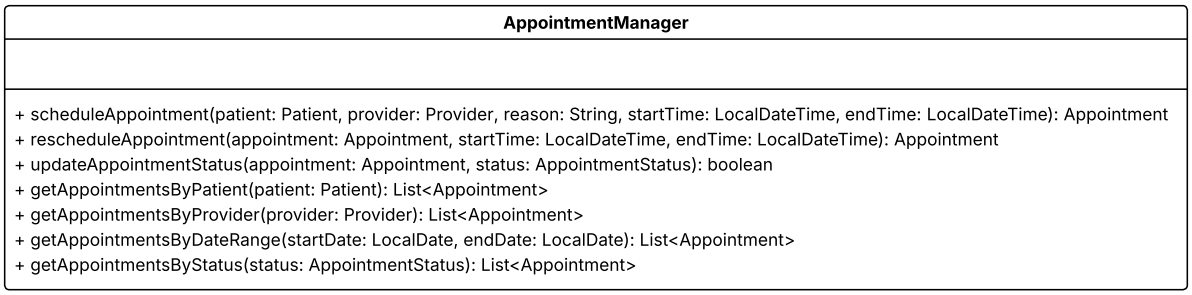

Figure 4: AppointmentManager class UML representation

The AppointmentManager class handles the primary scheduling logic of appointments including conflicts and implementation of business rules. While the other classes represent data objects, this class manages operations. Through various methods, AppointmentManager can:

- Create new appointments
- Reschedule existing appointments
- Update the status of appointments
- Search for appointments using different criteria

3 Explanation of Business Rules

When an appointment is scheduled or rescheduled, the AppointmentManager class checks several factors before allowing an appointment to be created or modified. The rules are enforced as follows:

Valid Appointment Times

Before creating an appointment, the AppointmentManager should verify:

- the start time is before the end time
- the appointment is not scheduled in the past
- the appointment falls within allowed business hours if your system uses them
- the appointment duration is valid, such as 15, 30, or 60 minutes
- required fields exist, such as:
  - patient
  - provider
  - date
  - start time
  - end time
  - status

Example Logic:

1. Reject if `startTime >= endTime`
2. Reject if `startDateTime < LocalDateTime (current)`
3. Reject if duration is duration is zero or negative

Check Provider Availability

The most important rule is that a provider cannot have two appointments at overlapping times. When a new appointment is requested, the system should query all appointments for that provider and compare times.
A conflict exists if:

- a new appointment starts during an existing one
- a new appointment ends during an existing one
- a new appointment completely contains an existing one
- Existing appointments completely contains the new one

Check Patient Conflicts

Optionally, the same logic can also be applied to patients so a patient cannot be booked into two appointments at the same time.
This prevents situations where:

- one patient is double booked with two providers
- One patient has overlapping appointments

The overlap check is the same as for providers.

Conflict Enforcement Logic

An appointment can only be created or updated if:

- its time range is valid
- it is not in the past (current `LocalDateTime` is before the proposed appointment time)
- it does not overlap with another appointment for the same provider
- its status is valid
- status changed follow allowed transitions

These rules are enforced through various methods (Refer to Figure 4) which handle the creation and manipulation of existing appointments. Each are described as follows:

`scheduleAppointment(...)` Method Logic

This method takes in various parameters including: `Patient, Provider, reason, startTime, endTime` and returns an `Appointment` object of the newly created appointment upon success.
This method should:

1. validate the appointment fields (verifying that the provided Patient and Provider) exist in the system
2. confirm the date and time are valid, and check whether the provider is free
3. optionally check whether the patient is free
4. assign a default status such as `SCHEDULED`
5. save the appointment only if all checks pass

If any rule fails, the appointment is not added and the system should return an error or throw an exception.

`rescheduleAppointment(...)` Method Logic

Rescheduling is similar to scheduling, except an existing appointment is being changed. This method takes in a few parameters in relation to the rescheduling logic of an appointment, including: `appointment, startTime (new)` and `endTime(new)` and returns a `Appointment` object of the rescheduled appointment.
This method should:

1. find the appointment by `appointmentId`
2. verify that the appointment exists
3. verify it is allowed to be rescheduled
    - for example, if an appointment is of `CANCELLED` or `COMPLETED` status, it cannot be rescheduled
4. validate the new `startTime` and `endTime`
5. check provider conflicts
6. ignore the current appointment during conflict checking so it does not conflict with itself
7. update the time if all checks pass
8. when checking for overlaps during rescheduling, exclude the appointment being rescheduled from the comparison.

`updateAppointmentStatus(...)` Method Logic

This method changes the status of an appointment. It takes in an `appointment` object and `status` as parameters, and returns a `boolean` value, true on success.
Status values are defined in the `AppointmentStatus` enum class and include:

- `SCHEDULED`
- `COMPLETED`
- `CANCELLED`

Enforcement Rules:
Examples:

- a `COMPLETED` appointment should not go back to SCHEDULED
- a `CANCELLED` appointment's status cannot be changed to `COMPLETED`
- only future appointments can be `SCHEDULED` or `CANCELLED`
- only appointments that have already happened can become `COMPLETED`

So the logic should check that the requested status transition is valid before updating it.

Search Logic

All search methods should return lists of filtered appointments (`List<Appointment>`).

`getAppointmentsByPatient()` Getter Method

This method takes in a `Patient` object as a parameter and returns a list of all appointments where the `patientId` matches.

`getAppointmentsByProvider()` Getter Method

This method takes in a `Provider` object as a parameter and returns a list of all appointments where the `providerId` matches.

`getAppointmentsByDateRange()` Getter Method

This method takes in `startDate` and `endDate` parameters. The `startDate` must occur prior to the `endDate` to create a valid search range. This method will return a list of appointments whose date/time fall within the given range.

`getAppointmentsByStatus()` Getter Method

This method takes in a specified `AppointmentStatus` as a parameter and returns a list of appointments with the requested status.
Note that the above getter methods do not enforce scheduling rules, but are useful for conflict checking and reporting.

4 Initial Code Skeleton

This section outlines the initial code skeleton implemented to demonstrate translation between UML and code. The repository for this source code can be found at: https://github.com/ericnmt/oop-clinic-scheduler/tree/main/src

AppointmentManager Class Initial Code Skeleton

```java
import java.time.LocalDate;
import java.time.LocalDateTime;
import java.util.ArrayList;
import java.util.List;

public class AppointmentManager {

    // Temporary block for demo
    // HashMap may be utilized later on
    private List<Appointment> appointmentList;
    public AppointmentManager() {
        this.appointmentList = new ArrayList<>();
    }

    // Logic methods
    /**
     * Method to handle scheduling of appointments
     * @param patient is the Patient that the Appointment will be created with
     * @param provider is the Provider that the Appointment will be created with
     * @param reason is the reason for the Appointment
     * @param startTime is the start of the Appointment
     * @param endTime is the end of the Appointment (Must be AFTEER startTime
     * @return Appointment
     */
    public Appointment scheduleAppointment(Patient patient, Provider provider,
    String reason, LocalDateTime startTime, LocalDateTime endTime) {
        // TBA: Scheduling logic: validate time, provider availability
        return null;
    }

    /**
     * Method to handle the modification of appointment status
     * @param appointment is the Appointment that will be rescheduled
     * @param startTime is the start time of the new Appointment
     * @param endTime is the end time of the new Appointment
     * @return Appointment
     */
    public Appointment rescheduleAppointment(Appointment appointment,
    LocalDateTime startTime, LocalDateTime endTime) {
        // TBA: Rescheduling logic: conflict checks, status validation
        return null;
    }

    /**
     * Method to handle the rescheduling of appointments
     * Method to handle the
     * @param appointment is the Appointment's status that will be changed
```

```java
     * @param status is the new status of the Appointment
     * @return boolean, true if status is the same as new status
     */
    public boolean updateAppointmentStatus(Appointment appointment,
    AppointmentStatus status) {
        return false;
        // TBA: Validation for status changes
    }

    // Search Methods
    /**
     * Method to retrieve all Appointments by Patient
     * @param patient is the filter for the list of Appointments
     * @return Appointment
     */
    public List<Appointment> getAppointmentsByPatient(Patient patient) {
        return new ArrayList<>();
        // TBA: Proper return signature, filter logic (by patientId)
    }

    /**
     * Method to retrieve all Appointments by Provider
     * @param provider is the filter for the list of Appointments
     * @return List of Appointments
     */
    public List<Appointment> getAppointmentsByProvider(Provider provider) {
        return new ArrayList<>();
        // TBA: Proper return signature, filter logic (by providerId)
    }

    /**
     * Method to retrieve all Appointments by a given date range
     * @param startDate is the startDate of the range
     * @param endDate is the endDate of the range
     * @return List of Appointments
     */
    public List<Appointment> getAppointmentsByDateRange(LocalDate startDate,
    LocalDate endDate) {
        return new ArrayList<>();
        // TBA: Proper return signature, filter logic (by date range)
    }

    /**
     * Method to retrieve all Appointments by Status
     * @param status is the filter for the list of Appointments
     * @return List of Appointments
     */
    public List<Appointment> getAppointmentsByStatus(AppointmentStatus status) {
        return new ArrayList<>();
        // TBA: Proper return signature, filter logic (by status)
    }
}
```

`Appointment` Class Initial Source Code

```java
import java.time.LocalDateTime;
public class Appointment {
    private int appointmentId;
    private Patient patient;
    private Provider provider;
    private LocalDateTime startDateTime;
    private LocalDateTime endDateTime;
    private AppointmentStatus status;
    private String reason;

    /**
     * @param appointmentId of Appointment, unique
     * @param patient who will be seen
     * @param provider who will see patient
     * @param startDateTime of Appointment
     * @param endDateTime of Appointment
     * @param status of Appointment
     * @param reason of Appointment
     */
    // Constructor, all parameters must exist
    public Appointment(int appointmentId, Patient patient, Provider provider,
    LocalDateTime startDateTime, LocalDateTime endDateTime, AppointmentStatus status,
    String reason) {
        this.appointmentId = appointmentId;
        this.patient = patient;
        this.provider = provider;
        this.startDateTime = startDateTime;
        this.endDateTime = endDateTime;
        this.status = AppointmentStatus.SCHEDULED;
        this.reason = reason;
    }

    // Getters and Setters
    public int getAppointmentId() {
        return appointmentId;
    }

    public void setAppointmentId(int appointmentId) {
        this.appointmentId = appointmentId;
    }

    public Patient getPatient() {
        return patient;
    }

    public void setPatient(Patient patient) {
        this.patient = patient;
    }

    public Provider getProvider() {
        return provider;
    }
```

```java
    public void setProvider(Provider provider) {
        this.provider = provider;
    }

    public LocalDateTime getStartDateTime() {
        return startDateTime;
    }

    public void setStartDateTime(LocalDateTime startDateTime) {
        this.startDateTime = startDateTime;
    }

    public LocalDateTime getEndDateTime() {
        return endDateTime;
    }

    public void setEndDateTime(LocalDateTime endDateTime) {
        this.endDateTime = endDateTime;
    }

    public AppointmentStatus getStatus() {
        return status;
    }

    public void setStatus(AppointmentStatus status) {
        this.status = status;
    }

    public String getReason() {
        return reason;
    }

    public void setReason(String reason) {
        this.reason = reason;
    }

    // Methods
    public boolean updateStatus(AppointmentStatus newStatus) {
        this.status = newStatus;
        return true;
    }

    // REVISE: Enforce business rules for scheduling logic
    public boolean reschedule(LocalDateTime newStart, LocalDateTime newEnd) {
        this.startDateTime = newStart;
        this.endDateTime = newEnd;
        return true;
    }
}
```

`Patient` Class Initial Code Skeleton

```java
import java.time.LocalDate;

public class Patient{
    private int patientId;
    private String name;
    private LocalDate dateOfBirth;
    private String contactInfo;

    /**
     * Constructor, all parameters must exist
     * @param patientId of Patient
     * @param name of Patient
     * @param dateOfBirth of Patient
     * @param contactInfo of Patient
     */
    public Patient(int patientId, String name, LocalDate dateOfBirth, String contactInfo) {
        this.patientId = patientId;
        this.name = name;
        this.dateOfBirth = dateOfBirth;
        this.contactInfo = contactInfo;
    }

    // Getters and Setters
    public int getPatientId() {
        return patientId;
    }

    // REVISE: A patientId should always remain the same
    public void setPatientId(int patientId) {
        this.patientId = patientId;
    }

    public String getName() {
        return name;
    }

    public void setName(String name) {
        this.name = name;
    }

    public LocalDate getDateOfBirth() {
        return dateOfBirth;
    }

    public void setDateOfBirth(LocalDate dateOfBirth) {
        this.dateOfBirth = dateOfBirth;
    }

    public String getContactInfo() {
        return contactInfo;
    }

    // REVISE: A patient may have multiple methods of contact
```

```java
    public void setContactInfo(String contactInfo) {
        this.contactInfo = contactInfo;
    }

    // Patient toString method
    @Override
    public String toString() {
        return "Patient{" +
                "patientId='" + patientId + '\'' +
                ", name='" + name + '\'' +
                ", dateOfBirth='" + dateOfBirth + '\'' +
                ", contactInfo='" + contactInfo + '\'' +
                '}';
    }
}
```

Provider Class Initial Code Skeleton

```java
public class Provider{
    private int providerId;
    private String name;
    private String specialty;
    private String location;

    // Constructor, all parameters must exist
    /**
     * @param providerId of Provider, unique
     * @param name of Provider
     * @param specialty of Provider
     * @param location of Provider
     */
    public Provider(int providerId, String name, String specialty, String location) {
        this.providerId = providerId;
        this.name = name;
        this.specialty = specialty;
        this.location = location;
    }

    // Getters and Setters
    public int getProviderId() {
        return providerId;
    }

    // REVISE: A provider ID should never change after initially set
    public void setProviderId(int providerId) {
        this.providerId = providerId;
    }

    public String getName() {
        return name;
    }

    public void setName(String name) {
        this.name = name;
    }
```

```java
    public String getSpecialty() {
        return specialty;
    }

    public void setSpecialty(String specialty) {
        this.specialty = specialty;
    }

    public String getLocation() {
        return location;
    }

    public void setLocation(String location) {
        this.location = location;
    }

    // Provider toString method
    @Override
    public String toString() {
        return "Provider{" +
                "providerId=" + providerId +
                ", name='" + name + '\'' +
                ", specialty='" + specialty + '\'' +
                ", location='" + location + '\'' +
                '}';
    }
}
```

AppointmentStatus enum Class Initial Code Skeleton

```java
public enum AppointmentStatus {
    SCHEDULED,
    COMPLETED,
    CANCELLED
}
```

Main demo Code

```java
import java.time.LocalDate;
import java.time.LocalDateTime;
import java.util.ArrayList;
import java.util.List;

// Main method to test functionality of basic class code translations
public class Main {
    public static void main(String[] args) {
        System.out.println("Checkpoint 1: Demo Appointment Scheduling");

        // Initialize list for patients and providers
        List<Patient> patients = new ArrayList<>();
        List<Provider> providers = new ArrayList<>();
        // Initialize AppointmentManager
        AppointmentManager manager = new AppointmentManager();
```

```java
        // Create patient
        Patient patient1 = new Patient(101, "John Doe", LocalDate.of(2000,01,01),
        "johndoe@example.com");
        // Add patient1 to list and print patient1 details
        patients.add(patient1);
        System.out.println("Registered Patient: " + patient1.toString());

        // Create provider
        Provider provider1 = new Provider(701, "Dr. Smith", "Cardiology", "Room A-11");
        providers.add(provider1);
        // Add provider1 to list and print provider1 details
        System.out.println("Registered Provider: " + provider1.toString());

        // Schedule appointment
        System.out.println("Appointment schedule demo:");
        // Schedule appointment on: April 1, 2026: 9AM to 9:30 AM
        // start and end times will be handled by the AppointmentManager
        LocalDateTime start = LocalDateTime.of(2026, 4, 1, 9, 0);
        LocalDateTime end = LocalDateTime.of(2026, 4, 1, 9, 30);

        // Create Appointment object, utilize AppointmentManager schedule method
        Appointment newAppointment = manager.scheduleAppointment(patient1, provider1,
        "Checkup", start, end);

        if (newAppointment != null) {
            System.out.println("New Appointment successfully scheduled for "
            + newAppointment.getPatient().getName() + " with provider "
            + newAppointment.getProvider().getName() + " at "
            + newAppointment.getStartDateTime());
        } else {
            System.out.println("Failed to create new appointment
            (Implementation TBA, scheduleAppointment() returned null)");
        }
    }
}
```

Main demo Code Output

```
Checkpoint 1: Demo Appointment Scheduling
Registered Patient: Patient{patientId='101', name='John Doe', dateOfBirth='2000-01-01',
contactInfo='johndoe@example.com'}
Registered Provider: Provider{providerId=701, name='Dr. Smith', specialty='Cardiology',
location='Room A-11'}
Appointment schedule demo:
Failed to create new appointment (Implementation TBA, scheduleAppointment() returned null)

Process finished with exit code 0
```

Note that the new appointment failed because scheduling logic and business rules have not yet been implemented into the `AppointmentManager` class.